ZenCluster

Intelligent Mutual Fund Clustering and Discovery

A Mini-Project Report

Submitted for

Data Warehousing and Mining Laboratory

Submitted by:
31370 Sai Sudhish Nagireddy
31372 Yashraj Vaibhav Sane
Class / Division:TY III
Department of Computer Engineering
Pune Institute of Computer Technology, Pune

Academic Year: 2026–2027

Abstract

ZenCluster is a data-mining mini-project that groups mutual-fund-style records into clusters according to selected financial characteristics. Rather than assigning a predefined label to every record, it uses unsupervised learning to discover groups with similar patterns. These groups can make a dataset easier to explore by presenting records with comparable return-style attributes, volatility, expense ratio, assets under management (AUM), and risk characteristics.

The system follows a data-warehouse-style workflow: ingest a CSV dataset, validate and preprocess records, standardize numeric attributes, apply K-Means clustering, evaluate cluster quality, and expose results through a REST API. A React/ Vite frontend communicates with a Python FastAPI backend. Pandas is used for data manipulation and scikit-learn for preprocessing, clustering, and evaluation. Visual summaries and filters allow users to inspect the resulting groups.

The demonstration dataset contains synthetic records and is intended only for software and algorithm demonstration. It is not actual mutual-fund performance data and must not be used as investment advice. This report describes the problem, requirements, theory, architecture, mathematical model, implementation workflow, testing, limitations, and future extensions.

Keywords: Data Mining, Data Warehousing, Unsupervised Learning, K-Means, Standardization, Silhouette Score, FastAPI, React.

Table of Contents

1. Introduction
2. Problem Statement, Objectives and Scope
3. Requirements Analysis
4. Theoretical Background
5. System Design and Architecture
6. Algorithm and Mathematical Formulation
7. Implementation Details
8. Results, Evaluation and Testing
9. Limitations and Future Enhancements
10. Conclusion

References and Submission Checklist

1. Introduction

1.1 Background

Data warehousing and data mining convert raw data into organized information that supports exploration and decision-making. A data warehouse consolidates data into a consistent structure, while data-mining techniques identify patterns, relationships, and groupings. Clustering is useful when the dataset does not contain reliable target labels.

Mutual funds are commonly described using attributes such as historical returns, volatility, expense ratio, and fund size. Inspecting each attribute separately may not reveal the overall similarity between records. Clustering can summarize records into groups whose members are relatively similar under a chosen feature representation. A cluster might, for example, contain records with comparatively low volatility, while another contains records with greater variation in return-style values.

1.2 Project Overview

ZenCluster provides an interactive workflow for exploring such patterns. Users can load a CSV dataset, choose a cluster count, run K-Means, inspect cluster sizes and profiles, explore records, and export results. The backend separates data handling and modelling from the user interface, allowing the algorithm to be tested independently.

1.3 Motivation

- Demonstrate an end-to-end data-mining pipeline rather than only a standalone algorithm.
- Connect preprocessing, feature scaling, clustering, evaluation, API design, and visualization.
- Use a finance-related example that is understandable to students while keeping the data synthetic.
- Provide a practical basis for explaining how unsupervised learning differs from classification.

1.4 Intended Users

The intended users are students, instructors, and demonstrators who want to understand clustering and inspect groups of structured records. The prototype is educational; it is not a regulated financial tool, fund-rating service, or investment recommendation engine.

2. Problem Statement, Objectives and Scope

2.1 Problem Statement

Given a structured dataset of mutual-fund-style records and numerical attributes, develop a system that preprocesses the records and groups them into K clusters using K-Means. The application should summarize clusters, provide basic quality metrics, support record exploration, and make results accessible through a backend API and web dashboard.

2.2 Objectives

- Create a repeatable CSV ingestion and preprocessing pipeline.
- Handle missing numerical values and standardize selected features before clustering.
- Implement K-Means with a configurable number of clusters.
- Calculate inertia and silhouette score when valid.
- Present cluster sizes, profiles, and record details through a web interface.
- Expose data and model operations through REST endpoints.
- Validate the API and modelling workflow with automated tests.
- Document assumptions, results, limitations, and reproducibility steps.

2.3 Scope

The scope includes CSV ingestion, preprocessing, K-Means clustering, descriptive cluster profiling, visual exploration, filtering, and CSV export. The project uses a local development setup and synthetic sample data. It does not require a production database, live financial feeds, or external market-data APIs.

2.4 Expected Outcome

The expected outcome is a prototype in which a user starts the API and frontend, views a dataset summary, runs clustering, and inspects generated groups. Cluster labels are arbitrary identifiers: cluster 0 is not inherently better or worse than cluster 1. Meaning must be inferred from feature profiles.

3. Requirements Analysis

3.1 Functional Requirements

ID	Requirement	Description
FR-01	Load dataset	Read the CSV and expose summary information.
FR-02	Validate data	Check expected columns and prepare numeric fields.
FR-03	Preprocess	Impute missing numeric values and standardize selected features.
FR-04	Run clustering	Run K-Means for a supported K, such as 2–8.
FR-05	Evaluate	Return inertia and silhouette score when valid.
FR-06	Profile clusters	Summarize cluster sizes and typical feature values.
FR-07	Explore records	Search or filter records.
FR-08	Export results	Download clustering output as CSV.
FR-09	Dashboard	Show summaries, charts, profiles, and records.

3.2 Non-Functional Requirements

Category	Requirement
Usability	Clear dashboard, readable charts, simple controls.
Performance	Small sample dataset processes on a typical student laptop.
Reliability	Invalid input produces understandable errors.
Maintainability	Separate data loading, clustering, API routes, and UI.
Reproducibility	Use a fixed random seed where appropriate and document setup.
Portability	Run locally on macOS, Windows, or Linux with supported versions.
Data safety	Use synthetic data and do not present output as investment advice.

3.3 Hardware Requirements

- Laptop or desktop with a modern multi-core processor.
- At least 4 GB RAM; 8 GB or more is preferable for development tools.
- Free storage for project files and dependencies.
- Internet access for initial package installation.

3.4 Software Requirements

Software	Purpose
Python 3.10+ or supported project version	Backend and modelling
FastAPI and Uvicorn	REST API and local server
Pandas and NumPy	Data loading and numeric processing
scikit-learn	Imputation, scaling, K-Means, and metrics
Node.js and npm	Frontend dependency management
React and Vite	Interactive dashboard
Browser and VS Code	Development and testing

3.5 Dataset Description

The sample CSV is designed to contain 240 synthetic records. Fields include fund_name, category, risk_level, return_1y, return_3y, return_5y, volatility, expense_ratio, aum_cr, and sharpe_ratio. The actual feature list used for modelling must be confirmed from the implementation.

Attribute	Meaning	Potential role
fund_name	Synthetic record identifier	Display/search; normally excluded from distance calculations
category	Illustrative category	Description/filter; encode only if intended
risk_level	Illustrative risk label	Description; not a target label
return_1y/3y/5y	Return-style numeric features	Candidate clustering features
volatility	Variation/risk proxy	Candidate feature
expense_ratio	Expense percentage-style value	Candidate feature
aum_cr	AUM-style value	Candidate feature
sharpe_ratio	Risk-adjusted return-style value	Candidate feature

These records are synthetic. Values do not represent an actual fund, provider, or historical market period.

4. Theoretical Background

4.1 Data Warehousing and ETL

A data warehouse organizes data for analysis rather than transaction processing. The extract-transform-load (ETL) process extracts data from a source, transforms it into a consistent and useful form, and loads it into an analytical system. In ZenCluster, extraction reads the CSV; transformation checks schema, handles missing values, selects attributes, and scales values; loading makes prepared data available to the clustering pipeline and API. A local CSV is simpler than a full warehouse database but demonstrates core ETL stages.

4.2 Data Mining

Data mining applies computational and statistical methods to discover patterns. Tasks include classification, regression, association-rule mining, anomaly detection, and clustering. Classification learns from labelled examples; clustering seeks structure in unlabelled observations. Since the desired groups are not supplied as verified labels, clustering is appropriate.

4.3 Clustering

Clustering groups observations so that points within a cluster are relatively similar and points in different clusters are relatively dissimilar according to a distance measure. Results depend on feature selection, preprocessing, distance geometry, and algorithm. Clusters are not objective truths; a different feature set or scaling method can produce different groupings.

4.4 K-Means

K-Means is a partition-based unsupervised algorithm. The user specifies K, the number of clusters. The algorithm initializes K centroids, assigns each observation to the nearest centroid, recalculates each centroid as the mean of assigned observations, and repeats until convergence or an iteration limit.

4.5 Feature Scaling

Financial-style features use different units and ranges. AUM can be numerically much larger than an expense ratio. Without scaling, large-range features may dominate Euclidean distance. StandardScaler subtracts the feature mean and divides by its standard deviation so features are more comparable in scale.

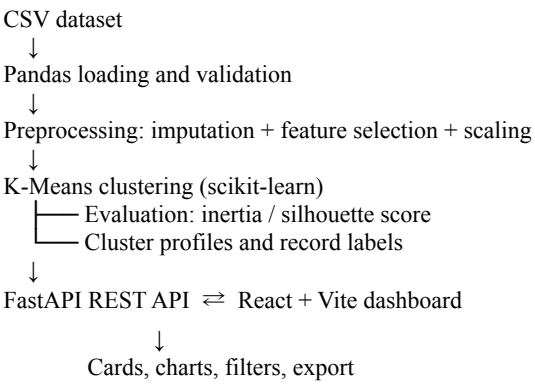
4.6 Evaluation Metrics

Inertia is the sum of squared distances between observations and their assigned centroids. It generally decreases as K increases, so it alone does not identify the best K. The elbow method looks for a point after which extra clusters provide diminishing improvement. The silhouette score compares average within-cluster distance with average distance to the nearest alternative cluster. It ranges from −1 to +1 when defined; higher values generally indicate better separation. Degenerate label configurations require special handling.

5. System Design and Architecture

5.1 Architecture Overview

ZenCluster uses a client-server architecture. React handles interaction and chart rendering, then sends HTTP requests to FastAPI. The backend reads the CSV, invokes preprocessing and clustering, computes summaries, and returns JSON responses. CSV export returns a downloadable tabular result.



5.2 Main Modules

Module	Responsibility
Data service	Loads CSV, validates columns, provides summaries and records.
Clustering service	Prepares numeric matrix, scales features, runs K-Means, calculates metrics.
API layer	Defines routes, validates parameters, returns JSON or CSV.
Frontend dashboard	Fetches data, renders charts/profiles, handles K selection and filters.
Test suite	Checks core endpoints, response shapes, and expected behaviour.

5.3 API Design

Illustrative endpoint	Method	Purpose
/api/health	GET	Check whether service is running.
/api/summary	GET	Return dataset size and summary statistics.
/api/cluster	GET or POST	Run clustering with requested K; verify actual method in code.
/api/clusters	GET	Return cluster sizes or profiles.
/api/funds	GET	Search/filter records.
/api/export	GET	Download cluster results as CSV.

These paths describe the intended API surface. Confirm exact route names and methods against the current FastAPI source before a live demo.

5.4 Data Flow

1. Frontend requests the dataset summary.
2. Backend reads the CSV and validates required columns.
3. Numeric features are imputed when necessary and standardized.
4. K-Means assigns each record to a cluster and produces centroids.
5. Backend calculates metrics and cluster profiles.
6. Frontend renders summaries, plots, and record-level data.
7. User can change K, filter records, and export results.

6. Algorithm and Mathematical Formulation

6.1 K-Means Objective Function

Let the dataset contain n observations $x_1, x_2, \ldots, x_n$ and let the requested number of clusters be K. K-Means partitions observations into $C_1, C_2, \ldots, C_k$ and computes centroid μ_j for each cluster. It minimizes the within-cluster sum of squared errors:

$$J = \Sigma_{j=1}^{K} \Sigma_{x_i \in C_j} ||x_i - \mu_j||^2$$

Here, μ_j is the arithmetic mean of points assigned to cluster j. Each point is assigned to the centroid with the smallest squared Euclidean distance.

6.2 Standardization

For a feature with mean μ and standard deviation σ, the standardized value of observation x is:

$z = (x - \mu) / \sigma$

Standardization is fitted and applied consistently for the clustering run. A zero-variance feature needs safe handling because division by zero is not meaningful.

6.3 Silhouette Score

For observation i, let a(i) be its average distance to points in its own cluster, and b(i) be the minimum average distance to points in another cluster. Its silhouette value is:

$s(i) = [b(i) - a(i)] / \max(a(i), b(i))$

The overall score is the mean of eligible observations. Values near +1 indicate good matching to the assigned cluster, values near 0 suggest overlap, and negative values may indicate misassignment. Interpret cautiously for small or uneven clusters.

6.4 Algorithm Steps

8. Read the CSV and validate required columns.
9. Select numeric features relevant to the goal.
10. Impute missing numeric values using a documented strategy such as median.
11. Standardize the selected features.
12. Initialize K centroids using K-Means++.
13. Assign each record to the nearest centroid.
14. Recalculate centroids as means of assigned records.
15. Repeat until convergence or the iteration limit.
16. Attach cluster labels and calculate metrics when valid.
17. Build cluster profiles and return results through the API.

6.5 Choosing K

K is a modelling choice. The elbow plot compares K with inertia; silhouette scores provide a complementary view of separation. A reasonable K should also yield interpretable profiles and avoid clusters that are too small to be useful. The range K = 2 to 8 is suitable for exploration, but the selected value should be justified by observed results rather than asserted without measurement.

7. Implementation Details

7.1 Backend

The backend uses FastAPI to define request handlers and provide interactive API documentation. Uvicorn runs the application locally. Pandas reads the CSV, while scikit-learn provides imputation, StandardScaler, KMeans, and silhouette_score. Feature names should be centralized so training, profiles, and chart responses remain consistent.

7.2 Frontend

The frontend uses React components built with Vite. It fetches summary and clustering responses, manages loading and error states, lets users choose K, and renders cluster sizes, profiles, and record-level information. Charts should include titles, axis labels, and legends.

7.3 Local Setup

Start the backend from the project root using a dedicated terminal:

```
cd backend
python3 -m venv .venv
source .venv/bin/activate
```

```
pip install -r requirements.txt
uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

In a second terminal, start the frontend:

```
cd frontend
npm install
npm run dev
```

The frontend commonly runs at http://localhost:5173 and API documentation at http://localhost:8000/docs. If port 8000 is occupied, check whether a previous backend process is running before starting another. The CSV must be placed at the path expected by the data-loading code; for the described layout this is backend/data/mutual_funds.csv.

7.4 Error Handling

- Report a clear error when the CSV is missing or unreadable.
- Validate K before fitting and reject values outside the supported range.
- Handle missing columns and non-numeric values explicitly.
- Avoid silhouette calculation when the labels make it invalid.
- Show a visible frontend connection error when the API is unavailable.

7.5 Reproducibility

Use a fixed random_state where supported and document the selected features. Record K, preprocessing method, dataset version, inertia, and silhouette score for each experiment. Label the data as synthetic in the interface and exported files.

8. Results, Evaluation and Testing

8.1 Expected Output

With both services configured correctly, the dashboard should display dataset summary statistics, cluster distribution, a visual representation of records, cluster profiles, and a searchable record list. Changing K should refresh the labels and metrics. Export should produce a CSV containing record information and cluster assignments.

8.2 Evaluation Procedure

18. Verify the CSV path, row count, required columns, and missing-value profile.
19. Run clustering for several K values in the supported range.
20. Record inertia and silhouette score for each valid run.
21. Inspect cluster sizes and feature summaries.
22. Check whether groups are interpretable and whether any cluster is unexpectedly tiny.
23. Repeat with the same seed to check reproducibility.

Do not report invented scores as experimental results. Fill this table using values returned by the running application.

K	Inertia	Silhouette score	Cluster sizes	Interpretation
2	Measure during run	Measure during run	Record during run	Compare broad groups
3	Measure during run	Measure during run	Record during run	Inspect profile differences
4	Measure during run	Measure during run	Record during run	Check separation and balance
5	Measure during run	Measure during run	Record during run	Compare added complexity

8.3 Test Cases

ID	Test	Expected result
TC-01	Call health endpoint	Successful health response.
TC-02	Load sample CSV	Summary returned with expected records.
TC-03	Run K-Means with valid K	Each record receives a cluster label.
TC-04	Request unsupported K	Clear validation error.
TC-05	Simulate missing numeric values	Preprocessing applies documented strategy.
TC-06	Remove required column	Clear schema error.
TC-07	Check model metrics	Finite inertia; silhouette only when valid.
TC-08	Search/filter records	Only matching records returned.
TC-09	Export results	Readable CSV includes cluster labels.
TC-10	Stop backend while UI open	UI shows recoverable connection error.

8.4 Discussion

K-Means is efficient for small numeric datasets and provides a clear baseline for a student project. Its usefulness depends on feature selection and scaling. Cluster labels are not meaningful categories by themselves; profiles must be inspected before naming groups. The silhouette score helps but does not replace domain interpretation. Since the records are synthetic, any apparent financial pattern is a property of generated data, not evidence about actual markets.

9. Limitations and Future Enhancements

9.1 Limitations

- Synthetic data does not reflect actual fund returns, fees, or risk.
- K-Means assumes groups can be represented by centroids and is sensitive to scaling and outliers.
- The algorithm requires K to be selected.
- Feature selection and encoding affect distance calculations.
- A local CSV pipeline is simpler than a production warehouse and lacks scheduled ingestion or historical snapshots.
- The prototype is educational and does not assess personal investment suitability.

9.2 Future Enhancements

- Integrate a verified, licensed dataset with source, date range, and data dictionary.
- Add a repeatable ETL pipeline with schema checks, duplicate detection, and data-quality reporting.
- Compare K-Means with hierarchical clustering and DBSCAN.
- Add PCA-based 2D projection for visualization.
- Add repeated-seed and cluster-stability evaluation.
- Persist experiment configurations and metrics in a relational database.
- Add authentication, audit logs, deployment configuration, and CI tests for a larger system.
- Improve accessibility, responsive layout, and downloadable experiment summaries.

9.3 Responsible Interpretation

Cluster membership describes similarity under selected variables; it does not predict future returns or prove that a fund is suitable for a person. Before using real financial data, the project needs source verification, date ranges, permissions, and prominent disclaimers. Investment decisions should not be based on the synthetic demonstration.

10. Conclusion

ZenCluster demonstrates how data preparation, unsupervised learning, API development, and interactive visualization can be combined into an end-to-end data-mining application. It ingests a structured CSV, preprocesses numeric attributes, applies K-Means, evaluates group quality, and exposes results through a React dashboard backed by FastAPI. The project connects ETL, feature scaling, centroid-based clustering, inertia, and silhouette analysis to an implementable software workflow.

The main learning outcome is understanding that clustering results depend on data quality, selected attributes, scaling, and K. Synthetic data makes the prototype reproducible and suitable for demonstration, but limits conclusions about real financial products. Verified data, broader validation, and stronger data engineering would support a more complete analytical platform.

References

1. J. MacQueen, “Some Methods for Classification and Analysis of Multivariate Observations,” Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability, 1967.

2. scikit-learn, K-Means and clustering performance evaluation: https://scikit-learn.org/stable/modules/clustering.html

3. scikit-learn, StandardScaler documentation: https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html

4. FastAPI documentation: https://fastapi.tiangolo.com/

5. React documentation: https://react.dev/

6. Pandas documentation: https://pandas.pydata.org/docs/

7. Vite documentation: https://vite.dev/

Submission Checklist

- Fill in student name, class/division, roll number, and academic year.
- Verify final API route names and feature columns against source code.
- Run the project and fill the evaluation table with actual values.
- Include screenshots of the dashboard, cluster distribution, metrics, and API docs if required.
- Clearly identify the dataset as synthetic demonstration data.